

1 Task 1 – Visualizing the floor plans

The first task was to load and visualize the input data for a few floor plans to get familiar with the dataset. Each building is represented by two NumPy arrays: `<id>_domain.npy` (a 512×512 grid with 5°C on load-bearing walls, 25°C on inside walls and 0 elsewhere) and `<id>_interior.npy` (a boolean mask marking the cells that lie strictly inside a room).

A small Python script (`src/01-visualize-floorplans/visualize_floorplans.py`) reads both files for each building, builds a colour-coded RGB image of the domain (dark grey for outside, blue for cold load-bearing walls, red for heated inside walls and white for free interior cells) and plots it next to the binary interior mask. Some representative results are shown in Figure 1.

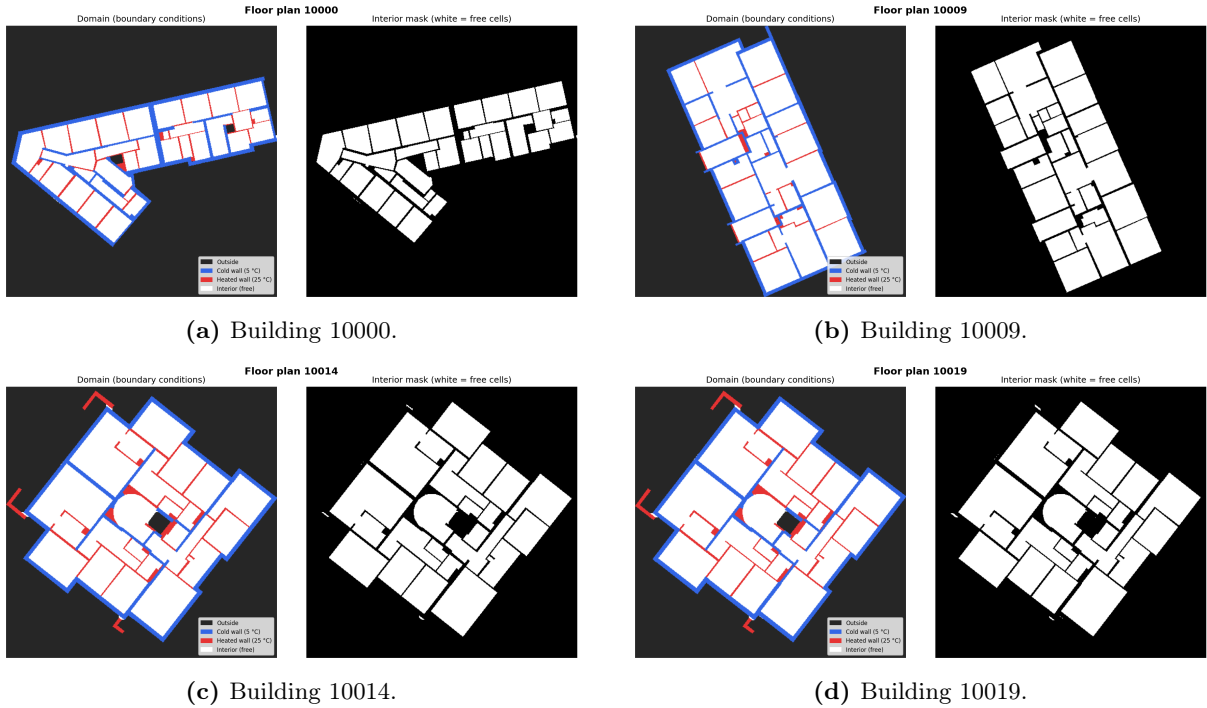


Figure 1: Example floor plans from the Modified Swiss Dwellings dataset. The left panel of each pair shows the colour-coded boundary conditions (red = inside walls at 25°C , blue = load-bearing walls at 5°C , white = interior). The right panel shows the corresponding binary interior mask.

2 Task 2 – Reference benchmark

The reference implementation in `simulate.py` was timed on a subset of $N = 10$ floor plans to estimate how long it would take to process the full dataset of 4571 buildings. The benchmark was executed as an LSF batch job on the DTU HPC `hpc` queue and repeated five times to obtain a reliable mean.

Each of the five repetitions ran in about 86 s, giving a mean of 86.255 ± 0.886 s for ten floor plans. This works out to ≈ 8.625 s per floor plan, so processing all 4571 floor plans serially with the reference NumPy implementation would take roughly

$$8.625 \text{ s} \times 4571 \approx 39,426 \text{ s} \approx 657 \text{ min} \approx 10.95 \text{ h.}$$

This is far too slow to be practical and motivates the optimisations performed in the rest of the project. Table 1 summarises the timing.

Quantity	Value
Subset size	10 floor plans
Number of repeated runs	5
Mean batch time	86.255 s
Standard deviation	0.886 s
Per-floor-plan time	8.625 s
Total floor plans available	4571
Estimated time for full dataset	39 426 s (≈ 10.95 h)

Table 1: Reference (NumPy) timing on $N = 10$ floor plans, run as an LSF batch job and averaged over five repetitions.

3 Task 3 – Visualizing the simulation results

Using the same reference solver, the steady-state temperature field was computed for the first three floor plans of the subset and visualised next to the input domain. Figure 2 shows three examples. As expected from Laplace’s equation, the temperature varies smoothly between the hot inside walls (25°C) and the cold load-bearing walls (5°C), with the warmest regions clustered around the inside walls and the coldest regions near the outer load-bearing walls. The exact same image format is used on the input domain (left) so that the boundary structure of each room can be compared visually with the resulting temperature distribution (right).

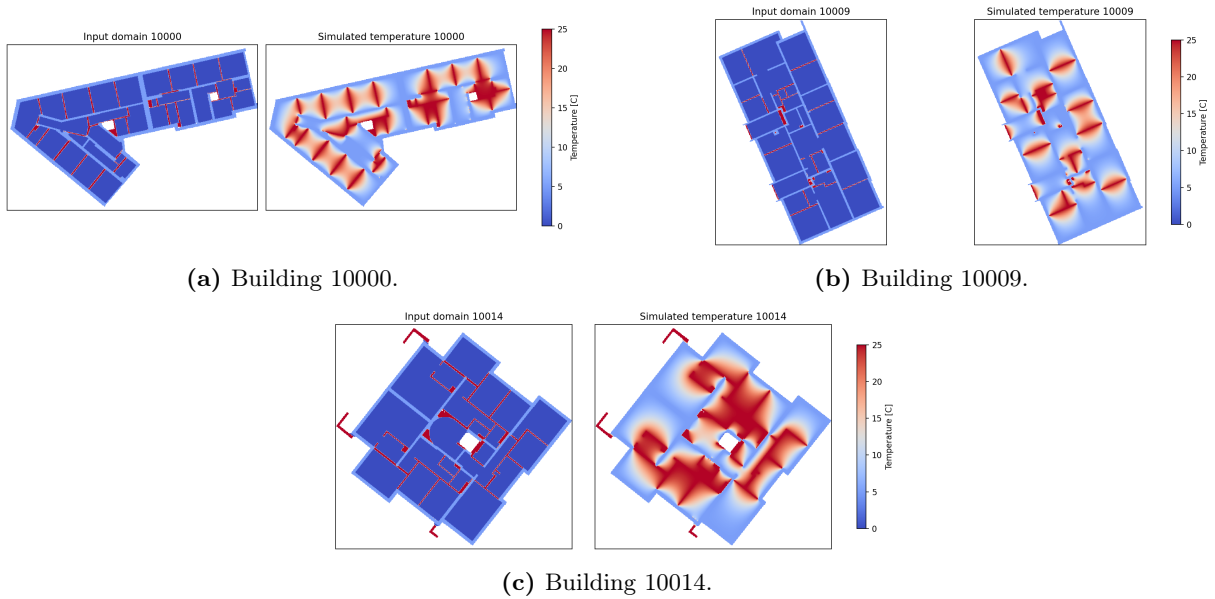


Figure 2: Input domain (left) versus simulated steady-state temperature (right) after the Jacobi solver has converged, for three example buildings.

The summary statistics for these three buildings (mean temperature, standard deviation, percentage of area above 18°C and below 15°C) will be revisited for the full dataset in Task 12.

4 Task 4 – Profiling the reference jacobi function

The reference `jacobi` function was profiled with `kernprof` (`line_profiler`) on a single floor plan, producing `outputs/04-profiling/simulate.py.lprof`. The function executes 3602 Jacobi iterations before converging, so per-line totals are dominated by the loop body. Table 2 reports the cumulative time spent on each significant line, expressed in seconds and as a percentage of

total jacobi time (5.30 s).

Code section (per iteration)	Total time (s)	% of total
<code>u_new = 0.25 * (u[1:-1, :-2] + u[1:-1, 2:] + ...)</code>	3.246	61.2%
<code>delta = np.abs(...).max()</code> (convergence check)	0.897	16.9%
<code>u[1:-1, 1:-1][interior_mask] = u_new_interior</code>	0.617	11.6%
<code>u_new_interior = u_new[interior_mask]</code>	0.539	10.2%
<code>if delta < atol: break</code>	0.003	0.05%
Pre-loop <code>u = np.copy(u)</code>	0.002	0.04%

Table 2: Line-profiler breakdown of the reference `jacobi` function for one floor plan (3602 iterations, total 5.30 s).

Interpretation. The hot path is clearly the five-point stencil update ($\approx 61\%$), which builds the full `u_new` array on every iteration. The convergence check together with the two masked indexing operations consumes another $\approx 39\%$. All four costly lines run on every one of the 3602 iterations, while the early-stopping branch and the initial copy are essentially free. This profile motivates Tasks 7–10: replacing the NumPy stencil with a JIT-compiled or GPU kernel removes the array-allocation overhead, while batching the convergence check (Task 10) eliminates the only serialising piece of work in the loop body.

5 Task 5 – Static scheduling

A new entry point, `simulate_static_scheduling.py`, parallelises the work across floor plans using Python’s `multiprocessing.Pool`. Each worker independently loads its assigned buildings from disk, runs the Jacobi solver and returns the summary statistics, so no large NumPy arrays have to be pickled across process boundaries. Static scheduling is implemented by passing `chunksize = [N/P]` to `pool.map`, which gives every worker a contiguous block of floor plans.

The benchmark was run on $N = 100$ floor plans on the DTU `hpc` queue using a single node with up to 64 cores. The results are shown in Table 3 and Figure 3.

Workers P	Time [s]	Speed-up $S(P)$
1	763.63	1.00
2	419.01	1.82
4	213.95	3.57
8	144.47	5.29
16	79.81	9.57
32	54.40	14.04
64	43.20	17.68

Table 3: Static scheduling timing and speed-up on 100 floor plans.

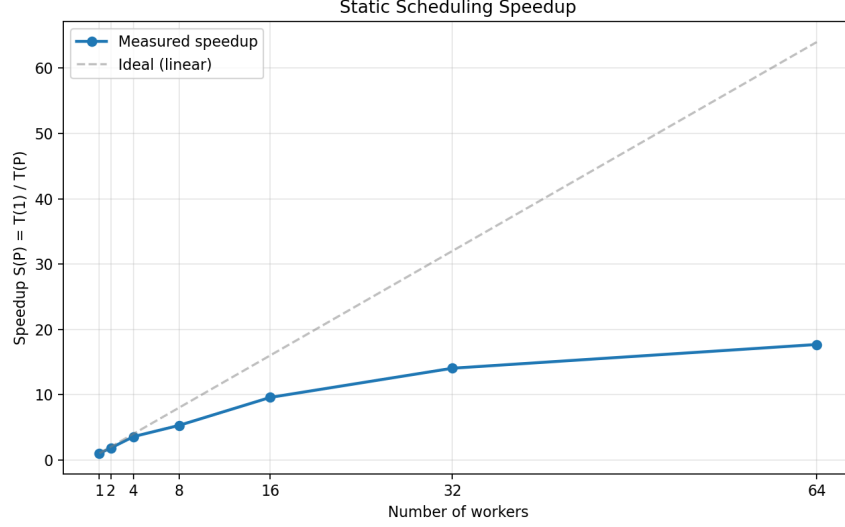


Figure 3: Measured speed-up of the static scheduling implementation versus the ideal linear scaling. The curve is sub-linear due to the serial fraction of the computation and to load imbalance across floor plans with different convergence times.

5b) Parallel fraction (Amdahl’s law)

Amdahl’s law states

$$S(P) = \frac{1}{(1 - f) + f/P}$$

which can be inverted to estimate the parallel fraction f from a measured speed-up:

$$f = \frac{1 - 1/S}{1 - 1/P}.$$

Substituting the best-measured speed-up $S = 17.68$ at $P = 64$ gives

$$f = \frac{1 - 1/17.68}{1 - 1/64} = \frac{0.9434}{0.9844} \approx 0.958.$$

Approximately **95.8%** of the computation is parallelisable.

5c) Theoretical maximum speed-up

$$S_{\max} = \frac{1}{1 - f} = \frac{1}{1 - 0.958} \approx 24.0.$$

The achieved speed-up of $17.68\times$ at $P = 64$ corresponds to about **73.5%** of the theoretical maximum. Diminishing returns are clearly visible past $P = 16$: doubling from 32 to 64 workers only improves the speed-up from $14.04\times$ to $17.68\times$ (a $1.26\times$ gain), indicating that the serial overhead and load imbalance dominate at higher worker counts.

5d) Estimate for the full dataset

Using the fastest configuration ($P = 64$, 43.20s for 100 floor plans) the per-floor-plan time is 0.432s, so the full dataset of 4571 buildings would take

$$0.432 \text{ s} \times 4571 \approx 1975 \text{ s} \approx 32.9 \text{ min}.$$

6 Task 6 – Dynamic scheduling

The same parallel benchmark was repeated with dynamic scheduling (`chunksize = 1`), so that whenever a worker finishes a floor plan it immediately picks up the next available one. The remaining setup is identical to Task 5.

6a) Did it get faster?

Table 4 compares the wall-clock times of static and dynamic scheduling for every worker count. Dynamic scheduling is consistently faster, with the largest absolute gain at $P = 8$ (43.85 s or roughly 30% faster). The improvement at $P = 1$ is small and within run-to-run variance, since both configurations execute sequentially.

Workers P	Static [s]	Dynamic [s]	Difference
1	763.63	734.73	baseline
2	419.01	367.17	51.84 s faster
4	213.95	191.69	22.27 s faster
8	144.47	100.62	43.85 s faster
16	79.81	58.50	21.31 s faster
32	54.40	38.82	15.58 s faster
64	43.20	37.40	5.80 s faster

Table 4: Wall-clock time per worker count for static vs dynamic scheduling on 100 floor plans.

The reason dynamic scheduling wins is **load balancing**: different floor plans require different numbers of Jacobi iterations to converge, so a static block can be dominated by a single slow building while other workers sit idle. With `chunksize = 1` the queue keeps every worker busy until the very last building has been dispatched.

6b) Did the speed-up improve?

Workers P	Static speed-up	Dynamic speed-up
1	1.00	1.00
2	1.82	2.00
4	3.57	3.83
8	5.29	7.30
16	9.57	12.56
32	14.04	18.93
64	17.68	19.65

Table 5: Speed-up of static vs dynamic scheduling.

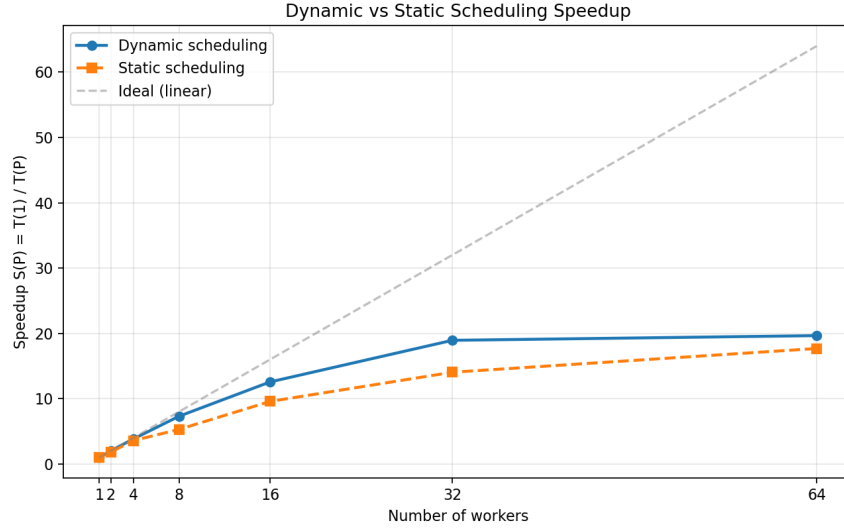


Figure 4: Speed-up curves for dynamic and static scheduling versus the ideal linear scaling. Dynamic scheduling is closer to linear at every worker count and the gap to static widens as more workers are added.

The speed-up improved across the board, and the gap widens with more workers. At $P = 32$ dynamic reaches $18.93\times$ versus static’s $14.04\times$, a substantial gain. At $P = 64$ dynamic reaches $19.65\times$ versus $17.68\times$, but the gain flattens, indicating that the remaining bottleneck is no longer load imbalance but the unavoidable serial overhead (process startup, I/O, result aggregation).

7 Task 7 – Numba JIT on the CPU

In Task 7 the `jacobi` function was rewritten as a Numba JIT-compiled kernel that runs on a single CPU core (`simulate_numba_jit_cpu.py`). The pure-Python solver uses two explicit nested loops (outer over rows i , inner over columns j) and applies the five-point stencil directly:

```

1 @numba.njit
2 def jacobi_numba(u, interior_mask, max_iter, atol=1e-6):
3     u = u.copy()
4     u_new = u.copy()
5     for _ in range(max_iter):
6         delta = 0.0
7         for i in range(512):
8             for j in range(512):
9                 if interior_mask[i, j]:
10                     val = 0.25 * (u[i+1, j] + u[i+1, j+2]
11                                   + u[i, j+1] + u[i+2, j+1])
12                     d = val - u[i+1, j+1]
13                     if d < 0.0:
14                         d = -d
15                     if d > delta:
16                         delta = d
17                     u_new[i+1, j+1] = val
18     u, u_new = u_new, u
19     if delta < atol:
20         break
21     return u

```

7a) Timing and comparison to the reference

The benchmark was run on $N = 10$ floor plans on the DTU hpc queue (single core). Results are shown in Table 6 and Figure 5.

Metric	Value
Wall time (incl. JIT compile)	38.901 s
Internal time (post-warmup)	36.432 s
Per-floor-plan (internal)	3.643 s
Reference per-floor-plan (NumPy, Task 2)	8.625 s
Speed-up vs reference	2.37×

Table 6: Numba JIT CPU timing on $N = 10$ floor plans.

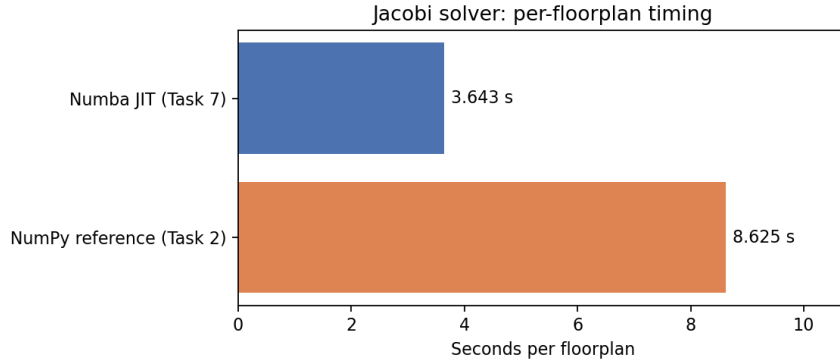


Figure 5: Per-floor-plan timing of the Numba JIT CPU solver versus the NumPy reference.

The Numba solution is $2.37\times$ faster than the reference. The one-time JIT compilation (~ 2.5 s, the difference between wall time and internal time) is amortised over the rest of the run.

7b) Function explanation and cache access pattern

The `@numba.njit` decorator compiles the function to native machine code and removes both the Python interpreter overhead and the temporary-array overhead of the NumPy stencil. Because NumPy arrays are stored in row-major (C-contiguous) order, iterating with `i` as the outer loop and `j` as the inner loop walks each row left-to-right one element at a time. All four stencil neighbours (`u[i+1, j]`, `u[i+1, j+2]`, `u[i, j+1]` and `u[i+2, j+1]`) come from rows `i`, `i+1` and `i+2`, which fit comfortably together in L2/L3 cache. Each inner-loop step advances one element along these three rows, so the access pattern is cache-line friendly with no strided or random accesses.

The convergence check is also kept inside the JIT-compiled region: a plain scalar `delta` is updated in place, avoiding any NumPy reduction call that would force Numba to drop back to the Python interpreter.

7c) Estimate for the full dataset

$$3.643 \text{ s} \times 4571 \approx 16,653 \text{ s} \approx 277.6 \text{ min} \approx 4.6 \text{ h.}$$

This is roughly $2.4\times$ faster than the NumPy reference estimate, but still impractical on a single core. Multi-core parallelism (Tasks 5–6) or GPU acceleration (Tasks 8–10) is needed to bring the full dataset within reach.

8 Task 8 – Custom Numba CUDA kernel

8a) Kernel and helper structure

Task 8 implements a custom CUDA kernel with Numba (`simulate_numba_cuda.py`). The kernel performs a *single* Jacobi iteration; the Python helper repeatedly launches it for a fixed number

of iterations, with no early-stopping criterion.

```

1 @cuda.jit
2 def _jacobi_kernel(u, u_new, interior_mask_u8):
3     i, j = cuda.grid(2)
4     if i < 512 and j < 512:
5         if interior_mask_u8[i, j] != 0:
6             u_new[i+1, j+1] = 0.25 * (u[i+1, j] + u[i+1, j+2]
7                                     + u[i, j+1] + u[i+2, j+1])

```

The launch configuration uses 16×16 -thread blocks covering the full 512×512 interior grid (256 threads per block, a common choice that keeps each block small enough for the streaming multiprocessors to schedule many concurrently). Each thread updates exactly one cell; non-interior cells are never written so the fixed boundary conditions are preserved automatically. The interior mask is stored as `uint8` rather than `bool` to avoid device-array instabilities seen on some CUDA driver versions.

The Python helper `jacobi_numba_cuda` owns two device buffers (`d_u`, `d_u_new`) and double-buffers them: after each kernel launch the references are swapped so the freshly written values become the read source for the next iteration. Both buffers are initialised from the input `u`, so the ghost rows used by the stencil are present from the start. The kernel boundary itself acts as a global barrier between iterations: no intra-kernel synchronisation is needed because each thread writes only to its own output cell.

8b) Timing and comparison to the reference

The benchmark was run twice on $N = 10$ floor plans on a DTU A100 GPU. Results are shown in Table 7.

Metric	Run 1	Run 2
Wall time (incl. CUDA JIT)	9.751 s	9.604 s
Internal time (post-warmup)	8.354 s	8.214 s
Per-floor-plan (internal)	0.835 s	0.821 s
Reference per-floor-plan	8.625 s	
Speed-up vs reference	10.3×	10.5×

Table 7: Numba CUDA timing on $N = 10$ floor plans (A100 GPU).

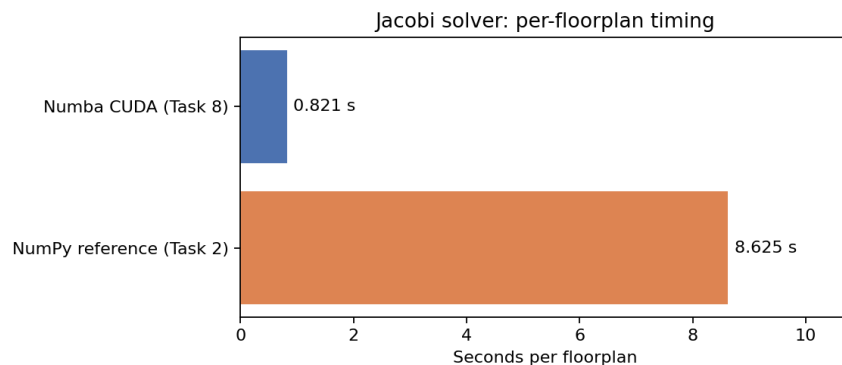


Figure 6: Per-floor-plan timing of the Numba CUDA solver versus the NumPy reference.

The two runs agree to within $\approx 1.7\%$, confirming that the timing is reliable. The Numba CUDA kernel is roughly **10.4×** faster than the NumPy reference, and the JIT-compilation overhead (≈ 1.4 s) is small.

8c) Estimate for the full dataset

$$0.821 \text{ s} \times 4571 \approx 3,754 \text{ s} \approx 62.6 \text{ min} \approx 1.0 \text{ h}.$$

This is a $4.4\times$ improvement over the Numba CPU solution from Task 7 and makes processing the full dataset in a single GPU job practical.

9 Task 9 – CuPy reference port

9a) Timing and comparison to the reference

In Task 9 the reference solver was ported to CuPy (`simulate_cupy.py`). The implementation mirrors the NumPy reference almost line-for-line, with `np` replaced by `cp`, but the convergence check still runs every iteration. Results on $N = 10$ floor plans on the A100 are shown in Table 8.

Metric	Value
Wall time (incl. CuPy JIT)	26.409 s
Internal time (post-warmup)	21.260 s
Per-floor-plan (internal)	2.126 s
Reference per-floor-plan	8.625 s
Speed-up vs reference	$4.1\times$

Table 8: CuPy timing on $N = 10$ floor plans (A100 GPU).

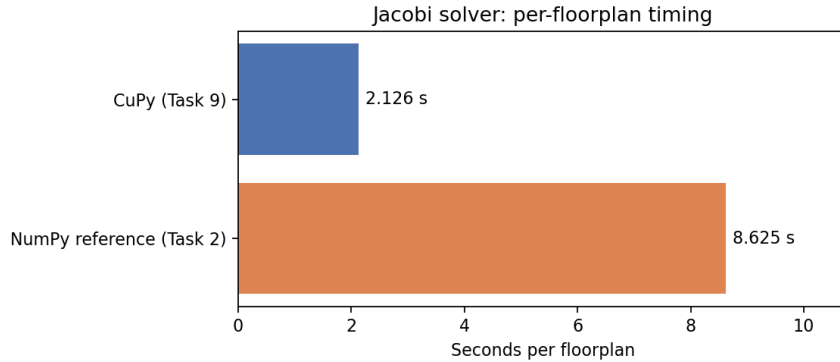


Figure 7: Per-floor-plan timing of the CuPy solver versus the NumPy reference.

The relatively large warmup overhead (≈ 5.1 s) is CuPy compiling its built-in GPU kernels on first use; the relevant figure is the post-warmup throughput.

9b) Estimate for the full dataset

$$2.126 \text{ s} \times 4571 \approx 9,718 \text{ s} \approx 162.0 \text{ min} \approx 2.7 \text{ h}.$$

9c) Surprising performance observation

The CuPy port is only $4.1\times$ faster than the NumPy reference, much slower than the custom Numba CUDA kernel ($10.5\times$, Task 8) on the *same* GPU, and even slower than a 64-core CPU run with dynamic scheduling ($19.65\times$, Task 6). This was surprising given that CuPy delegates all array operations to highly optimised cuBLAS/cuDNN kernels.

The bottleneck is the per-iteration convergence check:

```
1 delta = float(cp.abs(u_new[mask_gpu] - u_gpu[1:-1, 1:-1][mask_gpu]).max())
```

The `float(...)` call transfers a single scalar from GPU memory back to the CPU, which forces a **GPU→CPU synchronisation** on every iteration. With up to 20,000 iterations per floor plan this produces tens of thousands of pipeline stalls per building. The actual GPU arithmetic is fast; the synchronisation overhead dominates. Task 10 diagnoses this with `nsys` and fixes it by batching the convergence check.

10 Task 10 – Profiling CuPy with `nsys` and fixing the issue

The CuPy solver was profiled with NVIDIA’s `nsys` on a single building:

```
1 nsys profile --output nsys_original python simulate_cupy.py 1 <data_dir>
2 nsys stats nsys_original.nsys-rep
```

The relevant section of the report is the *CUDA API Summary (cudaapisum)*. For the original CuPy solver it showed `cuMemcpyDtoH_v2` (Device→Host copy) being called thousands of times (one per Jacobi iteration) and accounting for $\approx 95\%$ of total CUDA-API time. In contrast, the simple double-kernel example from the week 10 lecture only had two such calls. Each `cuMemcpyDtoH_v2` forces CuPy to flush all pending GPU work before copying the scalar to host memory: a full pipeline stall per iteration.

Fix: batched convergence check

Only perform the GPU→CPU transfer every `CHECK_INTERVAL = 100` iterations, reducing the number of syncs from up to 20,000 down to ≈ 200 per building:

```
1 if i % check_interval == check_interval - 1:
2     delta = float(cp.abs(u_new[mask_gpu] - u_gpu[1:-1, 1:-1][mask_gpu]).max())
3     converged = delta < atol
4 else:
5     converged = False
6
7 u_gpu[1:-1, 1:-1] = cp.where(mask_gpu, u_new, u_gpu[1:-1, 1:-1])
8
9 if converged:
10     break
```

δ is computed *before* the in-place update of `u_gpu`, so the convergence semantics match the original (the only side effect is that we may run up to 99 extra iterations past convergence, which is negligible). Re-profiling the optimised version with `nsys` confirms the diagnosis: `cuMemcpyDtoH_v2` now appears only ≈ 100 times and `cuLaunchKernel` dominates instead, which is the healthy profile we expect for a GPU-bound stencil computation.

Timing results

Variant	Per-floor-plan [s]	Speed-up vs Task 9
CuPy original (Task 9)	2.126	1.00×
CuPy optimised (Task 10)	0.828	2.57×
Numba CUDA (Task 8)	0.821	—

Table 9: Effect of the batched convergence check ($N = 10$ floor plans on an A100 GPU).

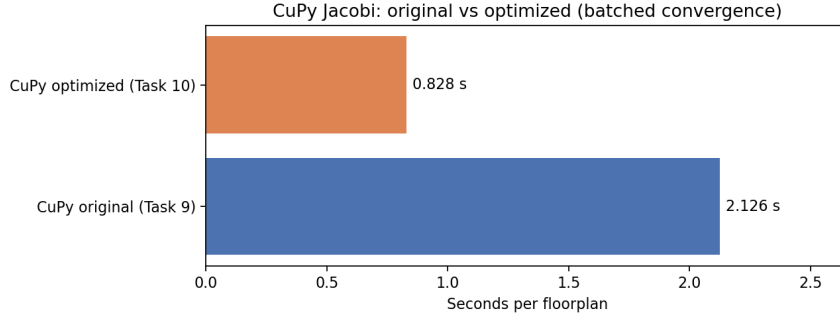


Figure 8: Per-floor-plan timing of the original (Task 9) versus optimised (Task 10) CuPy solver.

The optimised CuPy is **2.57** $\times$ faster than the original and matches the hand-written Numba CUDA kernel almost exactly (0.828 s vs 0.821 s). This confirms that the high-level CuPy array API is competitive with a custom CUDA kernel for this workload, once the per-iteration synchronisation is removed.

Estimate for the full dataset (optimised)

$$0.828 \text{ s} \times 4571 \approx 3,785 \text{ s} \approx 63.1 \text{ min.}$$

Note: mean temperatures from the optimised version differ very slightly from the original (e.g. 14.0131 vs 14.0123 for building 10000) because the batched check may run up to 99 extra iterations past convergence, marginally refining the solution. The deviation is well below the practical accuracy of the simulation.

11 Task 12 – Full dataset analysis

All 4571 buildings were processed with the optimised CuPy solver from Task 10. The per-building summary statistics were written to `outputs/12-full-analysis/all_results.csv` and analysed with Pandas in `src/12-full-analysis/analyze_results.py`.

12a) Distribution of mean temperatures

Figure 9 shows the four distributions of interest (mean temperature, temperature standard deviation, percentage of area above 18 °C and percentage of area below 15 °C), one histogram per building characteristic. The distribution of the mean interior temperature is centred around 14–15 °C with a long left tail of poorly heated buildings. Buildings with many hot inside walls cluster towards the higher end; buildings dominated by cold load-bearing walls cluster at the lower end.

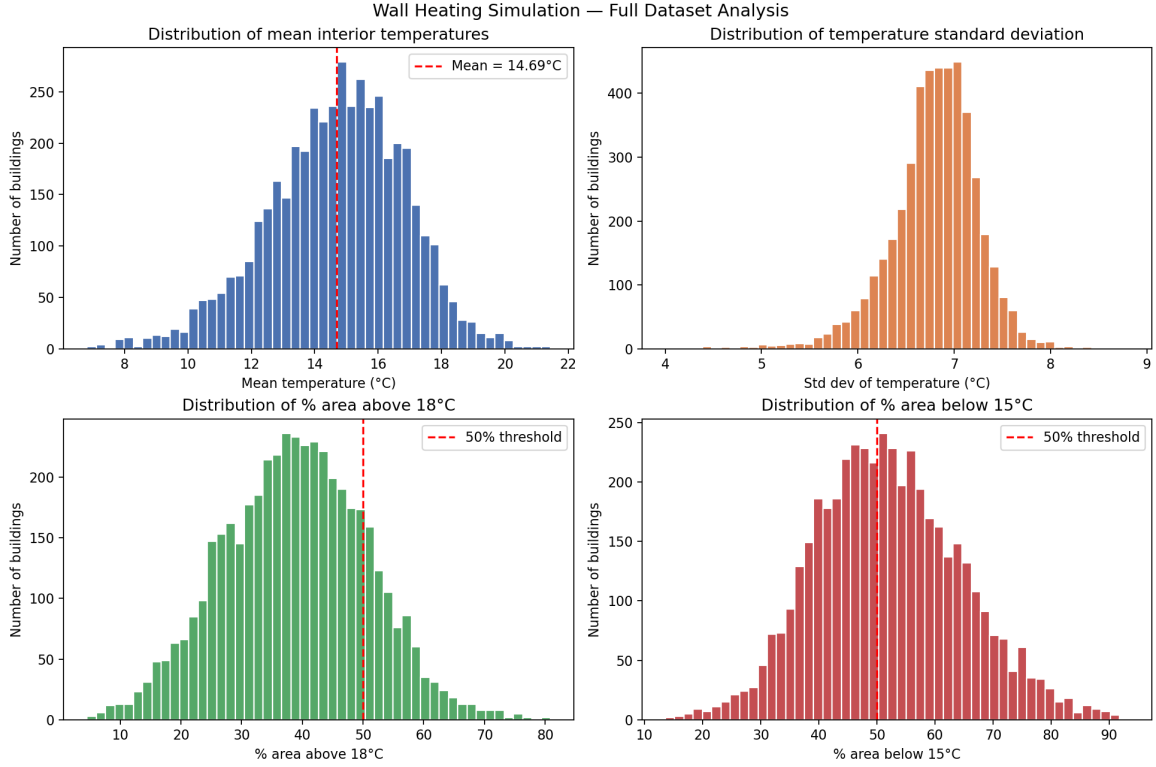


Figure 9: Distributions of the four per-building summary statistics over all 4571 floor plans (top-left: mean temperature; top-right: temperature standard deviation; bottom-left: % area above 18 °C; bottom-right: % area below 15 °C). The dashed vertical line in the bottom panels marks the 50 % threshold used in 12d) and 12e).

12b) Average mean temperature

$$\overline{\mu_T} = 14.6942 \text{ }^{\circ}\text{C}$$

12c) Average temperature standard deviation

$$\overline{\sigma_T} = 6.8016 \text{ }^{\circ}\text{C}$$

A larger standard deviation indicates more uneven heating: hot spots near inside walls combined with cold spots near load-bearing walls.

12d) Buildings with at least 50 % area above 18 °C

$$n_{>18} = 804 / 4571 \approx 17.6\%.$$

These are the well-heated buildings. The Wall Heating approach is most effective in floor plans with extensive inside-wall coverage relative to floor area.

12e) Buildings with at least 50 % area below 15 °C

$$n_{<15} = 2471 / 4571 \approx 54.1\%.$$

More than half of the dataset has the majority of its interior at uncomfortably cold temperatures. These are typically buildings with few inside walls (most walls are load-bearing) or with large open rooms whose centres are far from any heated wall.

Verdict on Wall Heating

Across the full dataset, Wall Heating only manages to keep at least half of the interior above 18 °C in about one in six buildings, while more than half of the buildings end up with the majority

of their area below 15 °C. With load-bearing walls fixed at 5 °C, the heated inside walls simply cannot push enough energy through the rooms to overcome the cold boundary, especially in floor plans with large open spaces or sparse inside-wall coverage. As an experimental retrofit replacing radiators, Wall Heating in its current form is therefore not a viable solution for most of the buildings in the Modified Swiss Dwellings dataset.

Summary of optimisations

Table 10 consolidates the per-floor-plan timing of every solution and the corresponding estimate for the full dataset of 4571 buildings.

Solution	Per-floor-plan [s]	Speed-up vs ref.	Full dataset
Reference NumPy (Task 2)	8.625	1.0×	10.95 h
Numba JIT, single CPU core (Task 7)	3.643	2.4×	4.6 h
Static scheduling, 64 cores (Task 5)	0.432	20.0×	32.9 min
Dynamic scheduling, 64 cores (Task 6)	0.374	23.1×	28.5 min
CuPy reference port (Task 9)	2.126	4.1×	162.0 min
Numba CUDA kernel (Task 8)	0.821	10.5×	62.6 min
CuPy optimised, batched check (Task 10)	0.828	10.4×	63.1 min

Table 10: Per-floor-plan timing and full-dataset estimate for every implementation explored in this project.

The dynamic-scheduling CPU solution remains the fastest end-to-end configuration overall thanks to its 64-fold parallelism, while the optimised CuPy and Numba CUDA kernels deliver the best single-device throughput and only need a single GPU to bring the full dataset down to about an hour.